

## **Task 1: Project Management**

### **Part: E (Individual report)**

For this project I was allocated to work on “USU Student App” subsystem.

My project management contribution was smaller compared to others, but I stayed on top of the deadlines we established as a group for our continuous progression and assisted anyone who needed help with Papyrus throughout the project completion.

I assisted in making the shared documentation as well as contributed toward shared diagrams. I always provided my insight on the others’ work when they wanted it. I have also helped to fix any inconsistencies in merging our diagrams.

I have made sure to stay consistent with working on the project as well as be open in my communicating with the rest of the team. I would notify the group when any major goals were completed on my part.

At one point in the very beginning, I realised that none of us are particularly good at using Papyrus so I decided to take it upon myself to learn the program so that I can assist anyone in need by explaining how to make what they want work within the environment of Papyrus.

**In conclusion**, I believe that I have done a good job at being a member of the team, the workload was big but manageable and the rest of the team did a great job at supporting me when I needed it.

## **Task 2: Analyse and Specify Software Quality Requirements**

The Student App must allow students to **register for an event** organised by a society or the university student union.

The student should be able to see the event and registration details, after which the system should confirm their status and notify them of it.

This requirement involves authentication, data retrieval, and writing registration data to the system’s backend.

Quality Requirements (QR):

## QR1: Security and Privacy Protection

### S1: Requiring Authentication

- Only logged in student may register for events
- Registration requests must include the student's secure session token

### S2: Data Privacy

- Personal data involved (such as student IDs, registration history, names) must be stored and transmitted using at least TLS 1.2 encryption.
- No sensitive data should appear in client-side logs or error messages

### S3: Access Control

- Students may only register themselves and should have no access to registering anyone else on their behalf
- The system must verify the student's eligibility (e.g. membership requirements, capacity limits)

### S4: Data Integrity

- Duplicate registrations must be caught and resolved using server-side tools
- Overbooking must be accounted for by checking the capacity at the moment of a booking attempt

## QR2: Performance Requirements

### P1: Registration Response Time

- The system should complete a registration request within 2 seconds under normal load

### P2: Efficient Database Access

- Queries for available event slots and user registration should be optimised to be processed in reasonable time.

### P3: Concurrent user support

- The system must support spikes in demand, such as when major university events open their registration windows

## QR3: Reliability Requirements

### R1: Guaranteed record of registration

- Once the server confirms the registration, the event slot must be securely stored with minimal risk of loss.

#### R2: Error Handling and Recovery

- If the app loses connection mid-registration the system must reset the registration process in order to avoid empty or incorrect registrations.

#### R3: Consistent State

- The system must ensure that the event capacity is updated correctly when multiple student register simultaneously

#### QR4: Scalability Requirements

##### SC1: Horizontal Scaling of Registration Service

- The backend registration API must allow for scaling across multiple servers or containers to handle increased user traffic

##### SC2: Database Scalability

- The event registration table must support rapid growth in number of events and users without performance degradation.

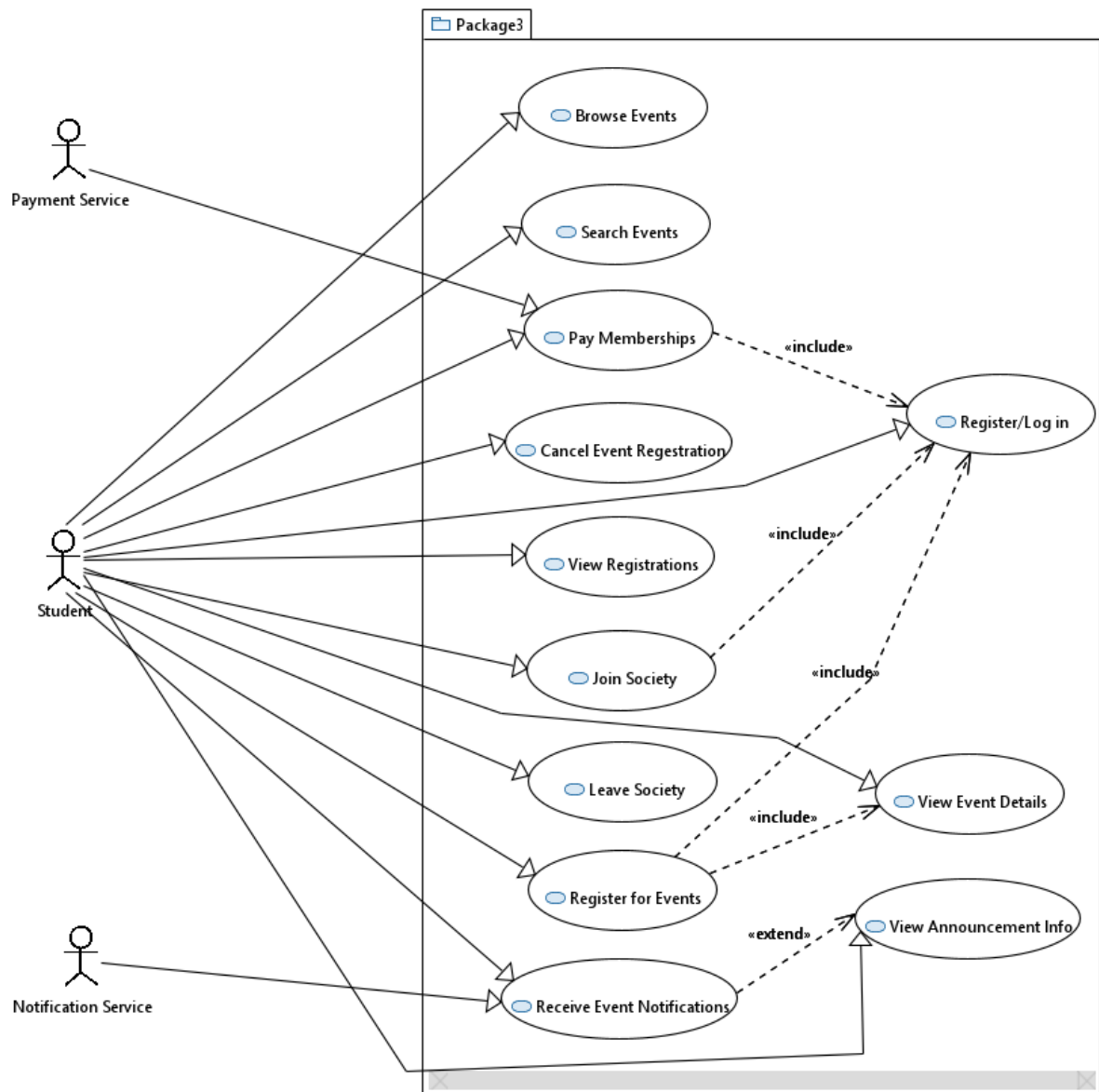
##### SC3: Load Spikes During Popular Events

- The system should support heavy bursts of concurrent attempts without crashing, queueing requests if needed.

**In conclusion**, security ensures only authorised students register, performance ensures fast and smooth interaction, reliability guarantees accurate and safe registration, and scalability ensures the system can support increasing users and event popularity.

### Task 3: Specification and Modelling Software Functional Requirements

a)



#### Task 4: Software Architectural Design

